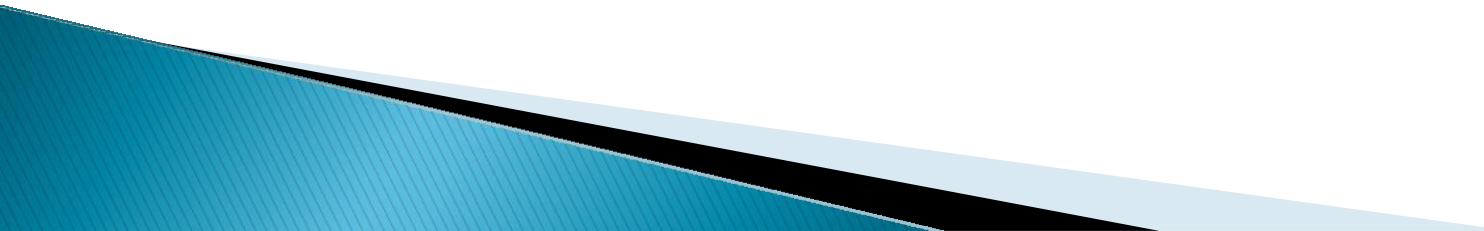
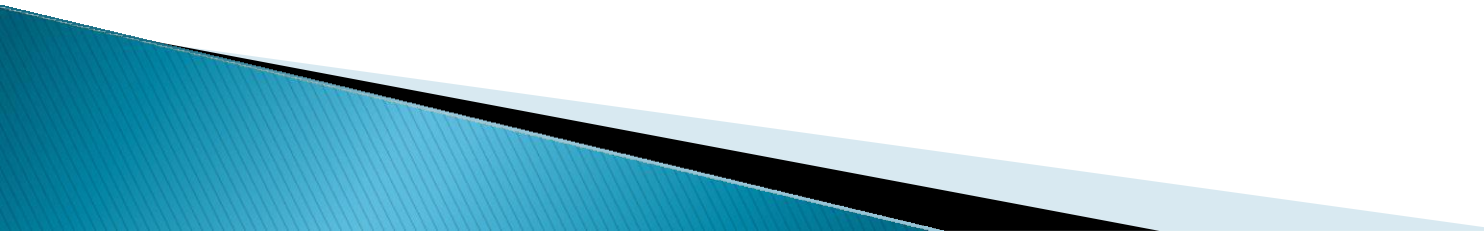


CSI315 – WEB TECHNOLOGY AND APPLICATIONS

PHP and MySQL database



Outline

- PHP database interfaces
 - Connecting to database
 - Creating and executing queries
- 

Documentation

- The PHP documentation contains complete descriptions of the PHP functions needed to communicate with MySQL
- A nice feature of PHP is that MySQL support is built-in.
- You can search the PHP docs for a complete description of all MySQL functions.
- <http://us3.php.net/manual/en/book.mysql.php>

PHP and database interfaces

- PDO (PHP Data Objects)
- MySQLi
 - Object oriented
 - Procedural
- For this course we will use MySQLi with OO interface.

Connecting To A Database

```
// Create connection  
$conn =new mysqli($servername, $username, $password,$dbname);
```

- connects to a MySQL server through a port
- username is a string for the user name
- password is a string for the password
- dbname is the name of the database, inside the server, that we are connecting to.
- returns FALSE on failure

Check connection success

```
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully <br /><br />";
```

- Recall: PHP object notation.
 - “Instance -> method/property” means invoke this method or access the value for this property on this object.
 - **\$conn** is an instance of **mysqli**, so we are retrieving value for property **connect_error**.
 - PHP Boolean class, in a Boolean test
 - 0, empty string, NULL, "" evaluate to **FALSE**
 - Any other number or non-empty string evaluate to **TRUE**
 - Therefore, if there is an error (returning error message or error code), the **if** condition evaluates to true.
 - **Die()** function prints a message and exits the current script.
 - Alias is **exit()**.

Check connection success

```
// Checkconnection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully <br /><br />";
```

□ For example:

Providing an incorrect password results in message below:

“Connection failed: Access denied for user 'rammidi'@'cs.ub.bw' (using password: YES)”

Close database connection

- The connection will be closed automatically when the script ends.
- To explicitly close the connection before, use command below:

```
// Close connection  
$conn->close();
```

- Note: Persistent database connections do not close automatically when script ends, hence allow re-use.

Complete Example

```
<?php
    $servername = "10.0.19.74";
$username = "usrnm";
$password = "passwd";
$dbname = "db_usrnm";
$conn = new mysqli($servername, $username, $password,$dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully <br /><br />";
$conn->close();
?>
```

- The above example will output:
- "Connected successfully" , assuming all login details correct.

Executing queries

- All queries require at least
 - A successful database connection.
 - An SQL statement.
- SQL query results
 - Create, insert, update do not return a resultset, therefore they return
 - TRUE if it executed successfully
 - FALSE, otherwise.
 - Select query returns a resultset, therefore use a variable to hold the results.
 - More efficient ways to handle results will be discussed later.

Executing queries

- General structure for executing create, insert, drop and update queries.

```
$sql = "...";    // Your SQLquery
if ($conn->query($sql) === TRUE)    //The feedback from
database will be equivalent to //TRUE if successful
{
    echo "Query executed successfully";
}
else {
    echo "Error executing query: " . $conn->error;
}
```

Sample table for examples

Table name: Add

Name	Type ?	Length/Values ?
dept_name	VARCHAR ▼	50
fac_name	VARCHAR ▼	50
HOD	VARCHAR ▼	50
Building	VARCHAR ▼	50

Create database

```
// Create database called, myUniversity

$sql = "CREATE DATABASE myUniversity";
if ($conn->query($sql) === TRUE){
    echo "Database created successfully";
} else {
    echo "Error creating database: " . $conn->error;
}
```

If you are going to create a database, provide only server name, username and password when creating a connection using `mysqli` object.

Create table

```
// Create table called, Department
// sql to create table
$sql = "CREATE TABLE Department (
    dept_name VARCHAR(50) PRIMARY KEY,
    fac_name VARCHAR(50) NOT NULL,
    HOD VARCHAR(50) NOT NULL,
    Building VARCHAR(50)
)";

if ($conn->query($sql) === TRUE){
    echo "Table Department created successfully";
} else {
    echo "Error creating table: " . $conn->error;
}
```

Insert record

```
$sql = "INSERT INTO Department (dept_name, fac_name, HOD, Building)
VALUES ('Computer Science', 'Science', 'Maz', 'Block 232')";
```

```
if ($conn->query($sql) === TRUE){
    echo "New record created successfully";
} else {
    echo "Error: " . $sql . "<br>" . $conn->error;
}
```

To obtain ID of last inserted record, necessary when using an

AUTO_INCREMENT field.

```
$last_id = $conn->insert_id;
```

SQL injection

Queries such as these

```
$sql = "INSERT INTO Department (dept_name, fac_name,  
HOD,Building) VALUES ('Computer Science', 'Science', 'Maz', 'Block  
232')";
```

Can be compromised by SQL injection attacks.

- https://www.w3schools.com/sql/sql_injection.asp
- Read about `mysqli_escape_string()`

Use Prepared Statements.

Note: All assessments in CSI315 will only accept prepared statements.

Prepared Statements and Bound Parameters

- Protects against SQL injection attacks.
 - Allows executing same or similar SQL statements multiple times, efficiently.
 1. Create an SQL template and send it to the database server. Leave some parameters unspecified, use ? as placeholders
 2. The database parses, compiles, and performs query optimization on the SQL statement template, and stores the result without executing it.
 3. Whenever it is required for execution, e.g. when customers complete and submit forms online, bind values to parameters and execute the query.
- 

Prepared Statements and Bound Parameters

Assume a scenario where we have a form that registers university departments.

- A MySQL database table "Department" with fields "**dept_name**" and "**fac_name**".

- // A successful database connection, **\$conn**

```
$conn = new mysqli($servername, $username, $password);
```

- An HTML form with the following fields:

```
<form action="/register.php">
```

```
<label >Department:
```

```
<input type="text" name="dept"><br></label><br>
```

```
<label Faculty: <input type="text" name="faculty"></label><br>
```

```
<input type="submit" value="Submit">
```

```
</form>
```

Prepared Statements and Bound Parameters

- An HTML form with the following fields:

```
<form action="/register.php">
```

```
<label>Department:
```

```
<input type="text" name="dept"><br></label><br>
```

```
<label Faculty: <input type="text" name="faculty"></label><br>
```

```
<input type="submit" value="Submit">
```

```
</form>
```

- Form data collected, validated into PHP variables: `$my_Dept` and `$my_Faculty`

Prepared Statements and Bound Parameters

□ // a) prepare and bind

```
$stmt = $conn->prepare("INSERT INTO Department  
(dept_name,fac_name)VALUES(? ,? )");
```

```
$stmt->bind_param("ss", $temp_Dept, $temp_Fac);
```

□ // b)set parameters and execute. Here we are assigning the values extracted from form: \$my_Dept and \$my_Faculty

```
$temp_Dept = $my_Dept;
```

```
$temp_Fac = $my_Faculty ;
```

```
if($stmt->execute())
```

```
{echo("record inserted");}
```

```
else {
```

```
    echo "Error inserting record: " . $stmt->error;
```

```
}
```

Select statement

```
//4. Now search records from database, using SELECT
$search="Accounts"; //We can assume this is data extracted from
form

$result = $conn->prepare("SELECT dept_name,fac_name FROM
Department where dept_name=?");
$result->bind_param("s", $temp_Dept);

$temp_Dept=$search;
$result->execute();

$result->bind_result($col1, $col2); //Binds returned results into
variables

while ($result->fetch()) {
    printf("%s %s\n", $col1, $col2);
}

$result->close(); //closes the resultset
```

Select statement

- Another useful method is `get_result()`. This is better than `bind_result()` as it allows to directly manipulate the resultset as an associative array, without binding the individual columns first.
- `$my_result = $result->get_result();`
- Requires more server configurations to enable.

Other SQL queries

- Other queries such as DELETE, UPDATE can also be implemented in a similar approach using PreparedStatement.